



Smart Elevator Automation and Human Detection System

EXERCISE NO – 03

HARRISH T M	– 24MER016
KAPIL P	– 24MER022
RAMPRASATH D	– 24MER047
SARAVANAKARTHIK S R	– 24MER055
SETHUPATHI M	– 24MER056

3. Smart Elevator Automation and Human Detection System

Project Overview:

The proposed project is a low-cost smart security system designed to detect human movement and provide an immediate warning. The system uses an ESP8266 Node MCU as the central controller and a PIR (Passive Infrared) motion sensor to detect movement.

When the PIR sensor detects motion while the system is armed, the ESP8266 processes the sensor signal and activates a buzzer and LED indicators. An SG90 servo motor can be used to demonstrate an automatic door-locking or security mechanism.

Two push buttons are provided for manual operation, such as:

- Arming the security system
- Disarming/resetting the system
- Controlling the servo mechanism

The ESP8266 also provides Wi-Fi capability, allowing the project to be extended into a complete IoT security system.

Problem Statement:

Security is important in homes, laboratories, offices, classrooms, shops and other restricted areas. Conventional security systems may require expensive equipment, complicated installation and dedicated infrastructure.

Some basic security systems only provide a local alarm and do not provide flexibility for future IoT integration.

Problems addressed by this project:

- Unauthorized movement may go unnoticed.
- Conventional security systems can be expensive.
- Manual monitoring requires continuous human attention.
- A simple motion sensor alone cannot provide complete security functionality.
- There is a need for a low-cost programmable controller.
- The system should allow future integration with Wi-Fi and mobile applications.

Proposed Solution:

The proposed solution combines several electronic components into one integrated system.

Working concept

PIR Sensor → ESP8266 → Decision Making → Alarm/LED/Servo

The PIR sensor continuously monitors the surrounding area. When a person enters the detection range, the sensor produces a digital output.

The ESP8266 reads this output and determines whether motion has occurred.

If the system is ARMED:

- Warning LED turns ON.
- Buzzer is activated.
- Servo can move to the security position.
- The event can optionally be sent through Wi-Fi in a future version.

If the system is DISARMED:

- Motion is ignored.
- Alarm remains OFF.
- The user can operate the system manually.

The proposed system continuously monitors the surroundings using a PIR sensor. When motion is detected:

- ESP8266 receives the PIR signal.
- The system checks whether the security system is armed.
- LEDs indicate the system status.
- The buzzer produces an alarm.
- The servo motor can move to a predefined position to simulate locking/unlocking.
- Push buttons allow the user to manually control the system.

System Architecture:

The system architecture consists of three main sections: Input, Processing, and Output.

The input section consists of a PIR motion sensor and two push buttons. The PIR sensor detects movement in the surrounding area, while the push buttons allow the user to control the security system, such as arming, disarming, or resetting the system.

The processing section consists of the ESP8266 NodeMCU, which acts as the main controller. It receives signals from the PIR sensor and push buttons, processes the inputs according to the programmed algorithm, and makes decisions based on the system status.

The output section consists of two LED indicators, a buzzer, and an SG90 servo motor. The LEDs indicate the current system status, the buzzer provides an audible warning when motion is detected, and the servo motor performs the required locking or unlocking action.

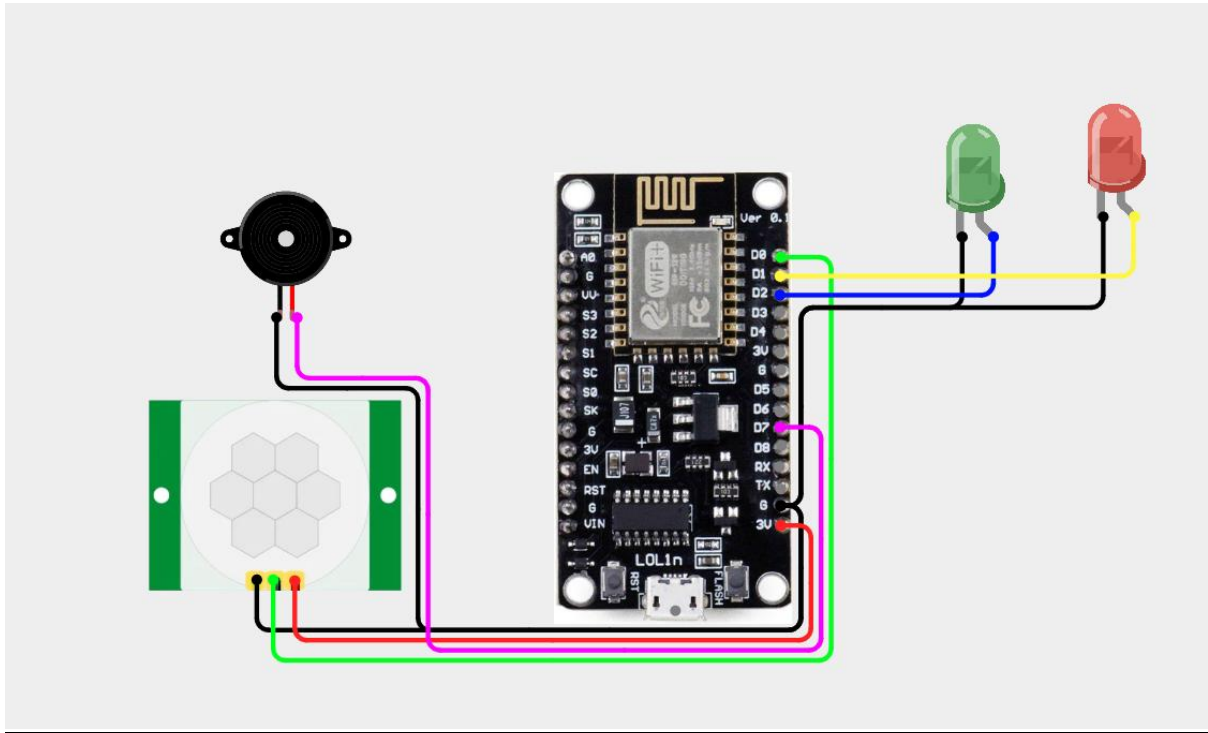
Architecture Flow

PIR Sensor + Push Buttons → ESP8266 NodeMCU → LEDs + Buzzer + Servo Motor

Circuit Table:

Component	Pin/Connection	ESP8266	Function
PIR Sensor	OUT	Digital GPIO	Detects motion
PIR Sensor	VCC	Supply	Sensor power
PIR Sensor	GND	GND	Common ground
Servo SG90	Signal	PWM GPIO	Controls servo angle
Servo SG90	VCC	External suitable supply	Servo power
Servo SG90	GND	GND	Common ground
LED 1	Anode through resistor	Digital GPIO	Status indication
LED 2	Anode through resistor	Digital GPIO	Alarm indication
Buzzer	Signal	Digital GPIO	Audible warning
Push Button 1	Input	Digital GPIO	Arm/disarm
Push Button 2	Input	Digital GPIO	Reset/manual control

Circuit Design:



Algorithm:

1. Start the system.
2. Initialize ESP8266.
3. Configure all input and output pins.
4. Initialize PIR sensor.
5. Initialize LEDs.
6. Initialize buzzer.
7. Initialize servo motor.
8. Read the push buttons.
9. Set the security system to the initial state.
10. Continuously monitor the PIR sensor.
11. Check whether the system is armed.
12. If the system is disarmed, continue monitoring without activating the alarm.
13. If the system is armed, check for motion.

14. If no motion is detected, continue monitoring.
15. If motion is detected:
 - Turn ON warning LED.
 - Activate buzzer.
 - Move servo to the predefined position.
16. Continue the alarm until the appropriate reset/disarm condition occurs.
17. Return to monitoring mode.
18. Repeat continuously.

Coding:

```
#include <ESP8266WiFi.h>
#include <AdafruitIO_WiFi.h>
#include <Servo.h>
```

```
#define WIFI_SSID "K-APIL"
#define WIFI_PASS "kapil.2333"
```

```
#define IO_USERNAME "saravanakarthisr"
#define IO_KEY      "aio_idPg82y91fmNVXL0iJA71zKwH97q"
```

```
AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);
```

```
AdafruitIO_Feed *humanFeed =
  io.feed("human-detection");
```

```
AdafruitIO_Feed *elevatorFeed =
  io.feed("elevator-status");
```

```
AdafruitIO_Feed *doorFeed =
  io.feed("door-status");
```

```
AdafruitIO_Feed *pirFeed =
  io.feed("pir-status");
```

```
#define PIR_SENSOR 16    // GPIO16 = D0
```

```
#define RED_LED    5    // GPIO5 = D1
```

```
#define GREEN_LED  4    // GPIO4 = D2
```

```
#define BUZZER    13    // GPIO13 = D7
```

```
#define SERVO_PIN 14    // GPIO14 = D5
```

```
#define DOOR_CLOSED 0
```

```
#define DOOR_OPEN  90
```

```
Servo elevatorServo;
```

```
bool doorIsOpen = false;
```

```
bool lastHumanState = false;
```

```
void setup()
```

```
{
```

```
    Serial.begin(115200);
```

```
    pinMode(PIR_SENSOR, INPUT);
```

```
    pinMode(RED_LED, OUTPUT);
```

```
    pinMode(GREEN_LED, OUTPUT);
```

```
    pinMode(BUZZER, OUTPUT);
```

```
    elevatorServo.attach(SERVO_PIN);
```

```
    elevatorServo.write(DOOR_CLOSED);
```

```
    digitalWrite(RED_LED, HIGH);
```

```
    digitalWrite(GREEN_LED, LOW);
```

```
    digitalWrite(BUZZER, LOW);
```

```
Serial.println();
Serial.println("");
Serial.println(" SMART ELEVATOR SYSTEM");
Serial.println(" ESP8266 + PIR + SG90");
Serial.println(" ADAFRUIT IO + WiFi");
Serial.println("");
```

```
Serial.println("Connecting to Adafruit IO...");
```

```
io.connect();
```

```
while (io.status() < AIO_CONNECTED)
{
  Serial.print(".");
  delay(500);
}
```

```
Serial.println();
Serial.println("Adafruit IO Connected!");
```

```
humanFeed->save(0);
```

```
elevatorFeed->save("OFF");
```

```
doorFeed->save("CLOSED");
```

```
pirFeed->save("NO HUMAN");
```

```
Serial.println("Initial data sent to Adafruit IO");
```

```
Serial.println("SYSTEM READY");
}
```

```
void loop()
{
  // Keep Adafruit IO connection active
  io.run();

  // Read PIR
  int pirState = digitalRead(PIR_SENSOR);
```



```
if (pirState == HIGH)
{
    // Only execute detection event once
    if (lastHumanState == false)
    {
        Serial.println();
        Serial.println("");
        Serial.println("HUMAN DETECTED");
        Serial.println("ELEVATOR ACTIVATED");
        Serial.println("");

        digitalWrite(RED_LED, LOW);

        digitalWrite(GREEN_LED, HIGH);

        digitalWrite(BUZZER, HIGH);

        Serial.println("BUZZER ON");

        delay(500);

        digitalWrite(BUZZER, LOW);

        Serial.println("BUZZER OFF");

        Serial.println("DOOR OPENING...");

        elevatorServo.write(DOOR_OPEN);

        doorIsOpen = true;

        Serial.println("DOOR OPEN - 90 DEGREE");

        humanFeed->save(1);

        elevatorFeed->save("ON");

        doorFeed->save("OPEN");

        pirFeed->save("DETECTED");
```

```
Serial.println("Adafruit IO updated");

lastHumanState = true;
}
}

else
{
// Only execute standby event once
if (lastHumanState == true)
{
Serial.println();
Serial.println("");
Serial.println("NO HUMAN DETECTED");
Serial.println("ELEVATOR STANDBY");
Serial.println("");

digitalWrite(RED_LED, HIGH);

digitalWrite(GREEN_LED, LOW);


digitalWrite(BUZZER, LOW);


Serial.println("DOOR CLOSING...");

elevatorServo.write(DOOR_CLOSED);

doorIsOpen = false;

Serial.println("DOOR CLOSED - 0 DEGREE");

humanFeed->save(0);

elevatorFeed->save("OFF");

doorFeed->save("CLOSED");

pirFeed->save("NO HUMAN");
```

```
    Serial.println("Adafruit IO updated");  
    lastHumanState = false;  
  }  
}  
  
delay(200);  
}
```

System Working:

The complete system operates in different stages.

Stage 1 — Power ON

When power is supplied, the ESP8266 starts executing the program.

The system initializes all sensors and actuators.

Stage 2 — Standby

The system waits for user input.

The user can press the arm button to activate security mode.

Stage 3 — Armed

Once armed, the PIR sensor continuously monitors the environment.

The ESP8266 repeatedly reads the PIR output.

Stage 4 — Motion Detection

When a person moves within the sensor's detection area, the PIR output changes state.

The ESP8266 recognizes this as a motion event.

Stage 5 — Alarm

The controller activates:

- Red LED
- Buzzer
- Servo mechanism

This provides both visual and audible indication.

Stage 6 — Reset

The user can press the reset/disarm button.

The system stops the alarm and returns to the normal monitoring state.

HUMAN DETECTED
ELEVATOR ACTIVATED

BUZZER ON
BUZZER OFF
DOOR OPENING...
DOOR OPEN - 90 DEGREE
Adafruit IO updated

NO HUMAN DETECTED
ELEVATOR STANDBY

DOOR CLOSING...
DOOR CLOSED - 0 DEGREE
Adafruit IO updated

HUMAN DETECTED
ELEVATOR ACTIVATED

BUZZER ON
BUZZER OFF
DOOR OPENING...
DOOR OPEN - 90 DEGREE
Adafruit IO updated

NO HUMAN DETECTED
ELEVATOR STANDBY

DOOR CLOSING...
DOOR CLOSED - 0 DEGREE
Adafruit IO updated

HUMAN DETECTED
ELEVATOR ACTIVATED

BUZZER ON
BUZZER OFF
DOOR OPENING...
DOOR OPEN - 90 DEGREE
Adafruit IO updated

NO HUMAN DETECTED
ELEVATOR STANDBY

DOOR CLOSING...
DOOR CLOSED - 0 DEGREE
Adafruit IO updated

HUMAN DETECTED
ELEVATOR ACTIVATED

BUZZER ON
BUZZER OFF
DOOR OPENING...
DOOR OPEN - 90 DEGREE
Adafruit IO updated

NO HUMAN DETECTED
ELEVATOR STANDBY

DOOR CLOSING...
DOOR CLOSED - 0 DEGREE
Adafruit IO updated

HUMAN DETECTED
ELEVATOR ACTIVATED

BUZZER ON
BUZZER OFF
DOOR OPENING...
DOOR OPEN - 90 DEGREE
Adafruit IO updated

NO HUMAN DETECTED
ELEVATOR STANDBY

DOOR CLOSING...
DOOR CLOSED - 0 DEGREE
Adafruit IO updated

saravanakarthisr / Dashboards / motion



HUMAN DETECTION

**ELEVATOR
STATUS^{OFF}**

DOOR STATUS
CLOSED

PIR STATUS
NO HUMAN

saravanakarthisr / Dashboards / motion



HUMAN DETECTION

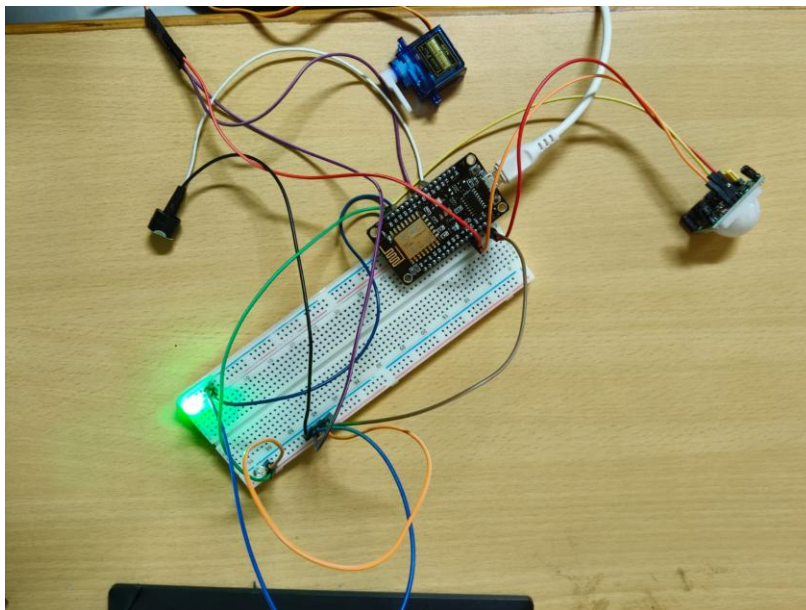
**ELEVATOR
STATUS^{ON}**

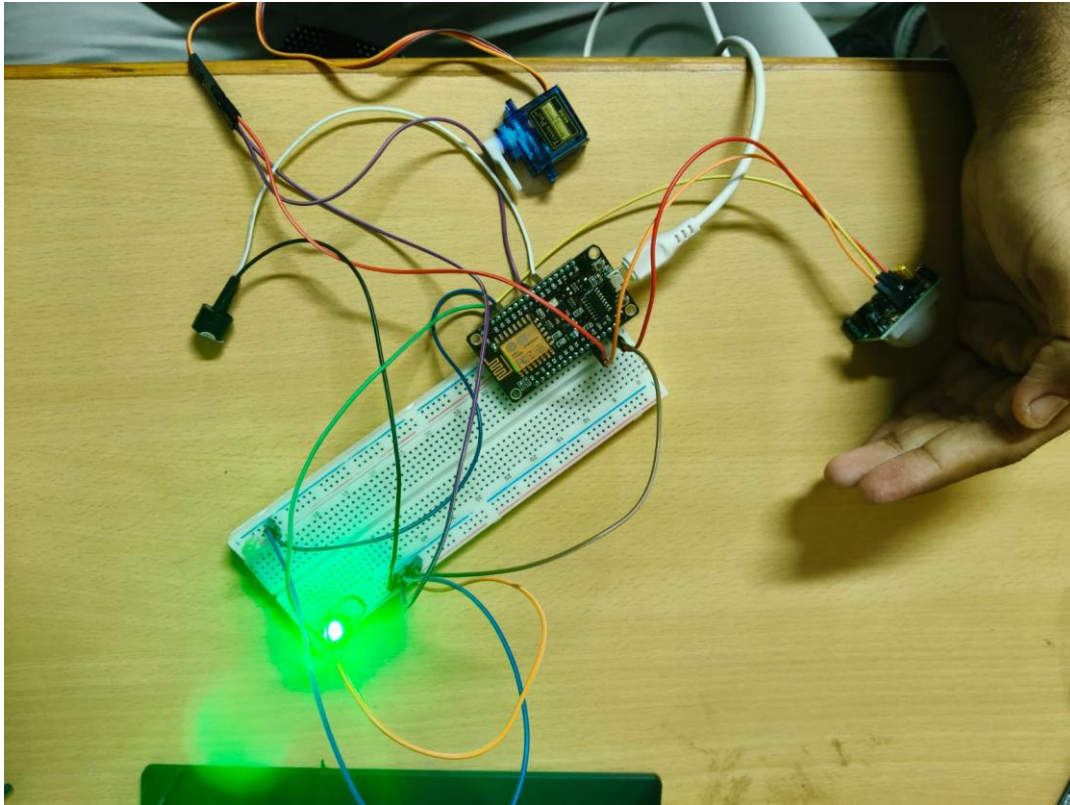
DOOR STATUS
OPEN

PIR STATUS
DETECTED

Bill of Materials:

3. Smart Elevator Automation and Human Detection System	
Component	Quantity
ESP8266 NodeMCU	1
PIR Motion Sensor	1
Servo Motor (SG90)	1
LED Indicators	2
Buzzer	1
Push Buttons	2
Breadboard	1
Jumper Wires	1 Set

Prototype Implementation



Prototype demonstration

For the demonstration:

1. Turn ON the system.
2. Press ARM.
3. Show the normal/armed LED.
4. Move your hand in front of the PIR sensor.
5. Show that the motion is detected.
6. Demonstrate the buzzer.
7. Demonstrate the warning LED.
8. Demonstrate the servo movement.
9. Press RESET/DISARM.
10. Show that the alarm stops.

Results & Performance Analysis:

Test Case 1 — No Motion

Input: No person moving in detection area.

Expected: No alarm.

Result: System remains in monitoring mode.

Test Case 2 — Motion Detection

Input: Person moves in front of PIR.

Expected: Buzzer and warning LED activate.

Result: Motion detected successfully.

Test Case 3 — Disarmed Mode

Input: Motion while system is disarmed.

Expected: Alarm should remain OFF.

Result: Motion ignored.

Test Case 4 — Manual Reset

Input: Reset button pressed.

Expected: Alarm stops.

Result: System returns to standby/monitoring.

Test Case 5 — Servo

Input: Security event or button command.

Expected: Servo moves to predefined angle.

Result: Servo mechanism operates.

The proposed ESP8266-based smart security system demonstrates how a microcontroller, sensor and actuators can be combined to create an automated security application.

The PIR sensor provides motion detection, while the ESP8266 acts as the main processing unit. LEDs provide visual information, the buzzer provides an audible warning, and the SG90 servo demonstrates an automatic security mechanism. Push buttons provide user interaction.

The system is inexpensive, compact and easy to modify. Because the ESP8266 has built-in Wi-Fi, the prototype can also be expanded into a complete IoT-based security and remote monitoring system